

Role	Mid-Level Flutter Engineer
Task	Currency Converter — Mobile App
Time Estimate	4 – 6 hours
Submission	GitHub link emailed to your recruiter
Deadline	As communicated by your recruiter

O V E R V I E W

What You Will Build

Build a Currency Converter mobile application using Flutter. The app fetches live exchange rates from a public API, allows the user to convert between currencies through a form, and persists conversion history locally on the device.

T A S K

Functional Requirements

Screens

- Converter screen — a form where the user enters an amount, selects a base currency and a target currency, and submits to see the converted result. The form must include proper input validation.
- History screen — lists all past conversions retrieved from local storage, displayed using a consistent, reusable list component.

API

- Fetch live exchange rates from a free public API
- Suggested: frankfurter.app (no API key required) or exchangerate-api.com (free tier)
- Loading and error states must be visible to the user

Local Storage

- Every completed conversion must be saved to the device
- Saved history must persist across app restarts

Design System

- Define a centralised theme — color palette, typography scale, and spacing tokens
- Build a small reusable widget library used consistently across all screens

(Optional)

- Unit or widget tests
- Offline fallback using last cached rates
- Dark mode support via the design system theme
- Ability to delete individual history entries

R E Q U I R E M E N T S

Technical Expectations

Apply Clean Architecture and a state management solution of your choice. Package and tool selection is your decision — we assess structure, not library choice.

- Flutter — latest stable channel
- State management — BLoC / Cubit preferred; Riverpod, MobX, accepted
- Responsive layout — functions correctly on phone and tablet
- No hard-coded secrets — store API keys securely if your chosen service requires one

A I U S A G E

Policy

AI tools are permitted and encouraged. Include a brief note in your README: which tools you used, what for, and how you verified the output. Engineering judgment and implementation quality are what we evaluate.

A clean, well-structured app outweighs a feature-rich one that is difficult to follow.

S U B M I S S I O N

How to Submit

- Push your work to a public GitHub repository
- README must include: setup instructions, architectural decisions, design system approach, and AI usage note
- Email the repository link to your recruiter before the stated deadline

The app must run from flutter run on a clean clone. Submissions that cannot be built will not be reviewed.

EFG Holding · Technology Division Questions? Contact your recruiter before you begin.